
NLP
ASSIGNMENT 1
Name: JIYA SINHA
Roll no: 22161

Question 01

Steps followed:

- **Loading and preprocessing:**

- Load the CSV file.
- Remove the rows with missing values in the ‘Sentence’ column.
- Preprocess text by:
 - * Converting it to lowercase
 - * Removing the URLs
 - * Change multiple dots to single dot
 - * Remove extra white spaces
 - * Remove non-alphabetical characters.

- **Feature extraction:**

N-gram features are extracted using the CountVectorizer with n-grams ranging from uni-grams to trigrams. The extracted features include:

- Unigrams
- Bigrams
- Trigrams

Example:

- **Unigrams:** [‘aa’, ‘aaa’, ‘aaaj’, ‘aaaleee’, ‘aaap’]
- **Bigrams:** [‘aa gae’, ‘aa gai’, ‘aa gaya’, ‘aa gaye’, ‘aa gayi’]
- **Trigrams:** [‘aa gae hai’, ‘aa gai ghar’, ‘aa gaya bharat’, ‘aa gaye bakwas’, ‘aa gaye hain’]

These features serve as inputs to the machine learning models.

- **Model training:**

We train a Naive Bayes classifier (MultinomialNB) on the extracted features. The training process includes:

- If there is a class imbalance, we can address it using RandomUnderSampler.
- We are performing hyperparameter tuning using GridSearchCV, with alpha values in the range [0.1, 0.5, 1.0, 5.0, 10.0].
- We are using 5-fold cross-validation with the F1-macro score for model selection.

- **Model Evaluation:**

The trained model is tested on the validation dataset. Evaluation metrics include:

	F1-score	
Dataset/Tags	0	1
Hate	0.62	0.53
Humor	0.49	0.73
Sarcasm	0.96	0.72

Table 1: F1-Scores

- Accuracy score.
- The classification report details the precision, recall, and F1-score.

- **Analysis:**

Challenges in Code-Mixed Classification Code-mixed text classification presents several challenges:

- **Limited Resources:** Limited resources are available for code-mixed languages.
- **Poor Annotation:** There might be annotation mistakes due to human-based annotation.
- **Spelling Variations:** Code-mixed texts often contain spelling variations.
- **No rules of grammar:** There do not exist any pre-defined rules of grammar or sentence formation, which leads to poor tokenization, POS tagging, and lemmatization, among other operations.
- **Feature Representation:** Traditional n-gram models may not fully capture the contextual meaning of the code-mixed text.

Improvement

- **New grammar rule:** Developing specialized tokenizers and lemmatizers that cater to code-mixed languages can improve pre-processing steps. These must handle cases such as spelling variations, and they should be able to tokenize, give POS tags, and lemmatize the text along with other tasks.
- **Multilingual Models:** Instead of using n-gram models and traditional ML models, we can use multilingual models such as multilingual BERT that are trained on large, multilingual corpora that can help capture the nuances of both languages involved in code-mixed text. These models can understand the context of both languages simultaneously and improve classification accuracy.

Question 02

- **HMM Model:**

- Create a set of vocabulary and POS Tags.
- Store the indices of each word and tags.
- Initialize tables to store:

-
- * **Start Probabilities:** A vector of size (number of tags) storing the count of each tag appearing at the beginning of a sentence.
 - * **Transition Probabilities:** A matrix of size (number of tags, number of tags) storing the count of transitions between tags.
 - * **Emission Probabilities:** A matrix of size (number of tags, number of words) storing the count of words emitted by each tag.
 - Apply k-smoothing by initializing all counts with alpha (user defined).
 - Normalize the tables to convert counts into valid probability distributions:
 - * **Start Probabilities:** Normalize each count by the total number of sentences.
 - * **Transition Probabilities:** Normalize each row so that the sum of probabilities for each previous tag is 1.
 - * **Emission Probabilities:** Normalize each row so that the sum of probabilities for each tag emitting words is 1.

• **Viterbi Algorithm:**

- Initialize variables:
 - * δ to store the highest probability of a tag at step $s+1$.
 - * ψ to store the best tag.
 - * A small probability value (10^{-4}) for unknown words to prevent zero probabilities.
- For the first word w_1 , the probability of each tag t is computed as:

$$\delta(t, 1) = P_{start}(t) \cdot P_{emission}(w_1|t)$$

If w_1 is unknown, a small probability is used instead of the emission probability.

- For each word w_j (from 2 to N), the algorithm computes:

$$\delta(t_j, j) = \max_{t_j} [\delta(t, j-1) \cdot P_{transition}(t_j|t) \cdot P_{emission}(w_j|t)]$$

- * If w_j is unknown, $P_{emission}(w_j|t)$ is replaced by the small probability value.
- * The tag that maximizes this probability is stored in ψ .
- Path is traced back to get the best tag sequence:

$$t_N = \arg \max_t \delta(t, N)$$

Testing :

- * For testing, we extract words and their true POS tags from the test data.
- * The most probable tag sequence for each sentence is predicted using viterbi algorithm.
- * We compute accuracy by comparing predicted tags with true tags.

$$Accuracy = \frac{CorrectPredictions}{TotalTags}$$

- * The model is evaluated on both clean (test_data.txt) and noisy test data (noisy_test_data.txt).

Result and analysis:

Smoothing (α)	Viterbi Accuracy (%)	Viterbi with Noise Accuracy (%)
No Smoothing ($\alpha = 0$)	89.64	81.31
$k = 0.5$	85.82	77.85
$k = 1$	83.45	75.74

Table 2: Viterbi Algorithm Accuracy

- The accuracy decreases when testing data with noise is used.
- The accuracy decreases with increasing smoothing, possibly due to over-fitting.
- Without smoothing, the model achieves the highest accuracy, suggesting that the training data sufficiently covers most cases.